Longest Increasing Subsequence

AlgoExpert · Very Hard · Dynamic Programming

The problem

- One strictly increasing subsequence of maximum length (patience sorting with predecessor links).

Intuition

- Patience sorting: keep the smallest possible tail for each length and binary search where each number goes.

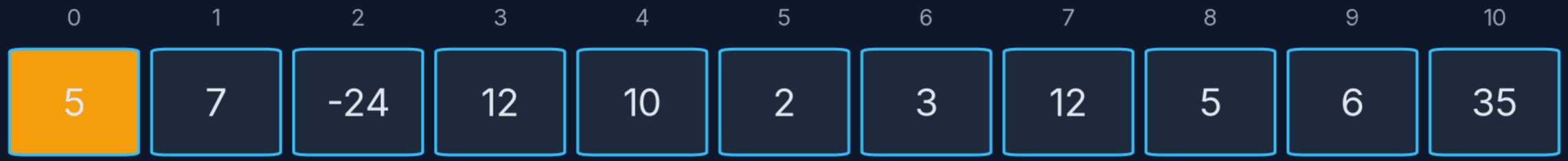
Brute force vs optimal

- DP: for each i check all j < i → $O(n^2)$
- Patience sorting tails + binary search → O(n log n)
- Prev pointers rebuild the sequence

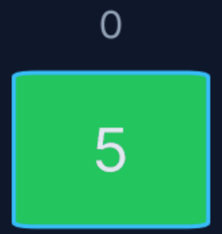
Approach

- 1. Tails array of indices
- 2. Binary search the position
- 3. Record predecessors
- 4. Rebuild from the last tail

Worked example

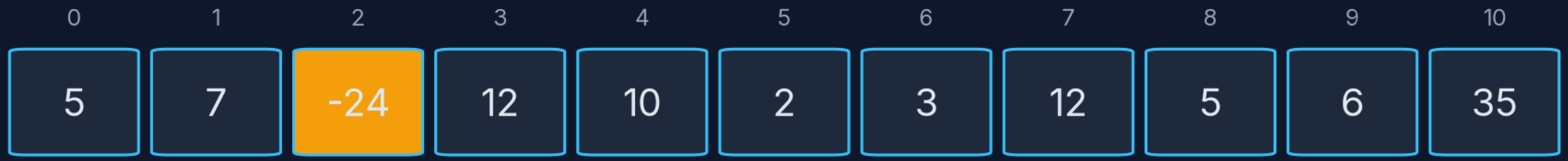


smallest tail for each length

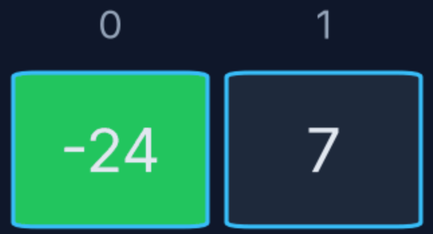


5 → tails[0]

Worked example

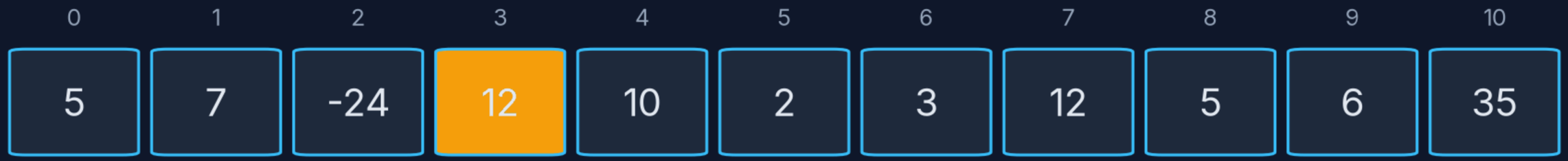


smallest tail for each length

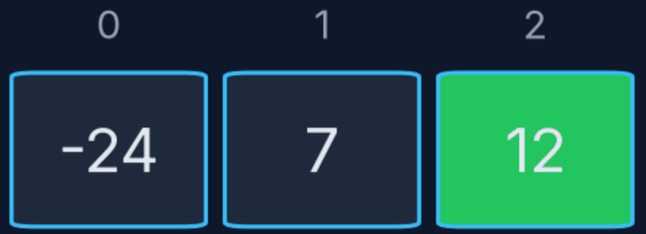


-24 → tails[0]

Worked example

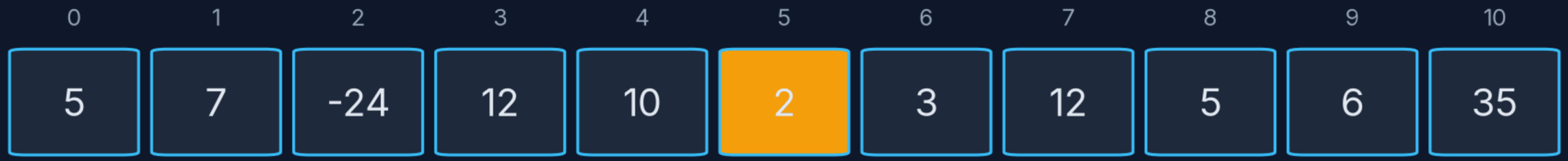


smallest tail for each length

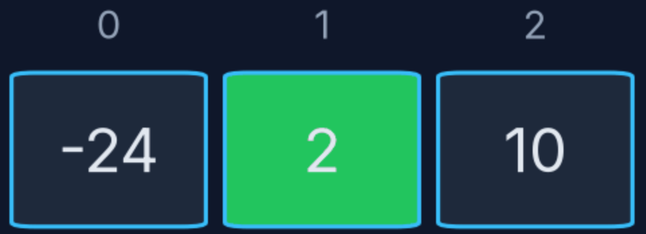


12 → tails[2]

Worked example

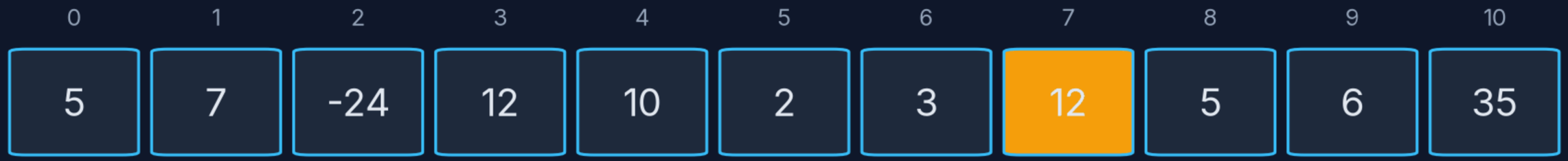


smallest tail for each length

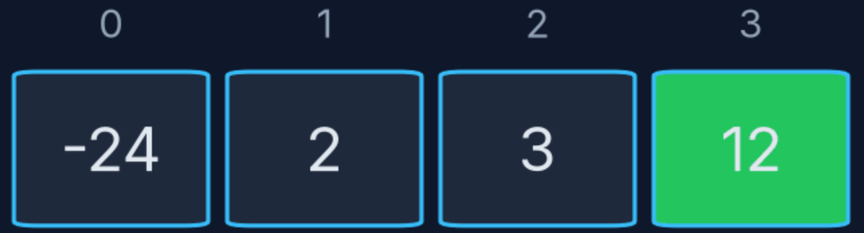


2 → tails[1]

Worked example

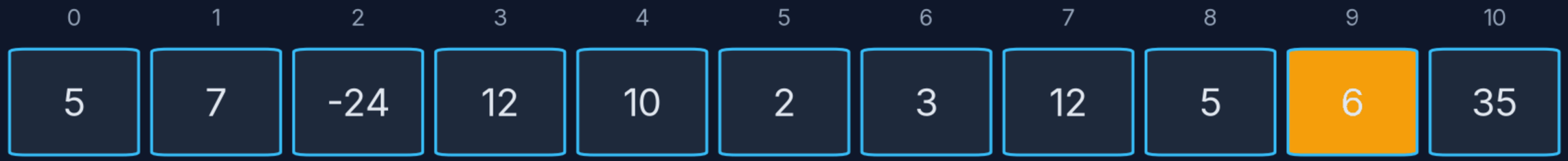


smallest tail for each length

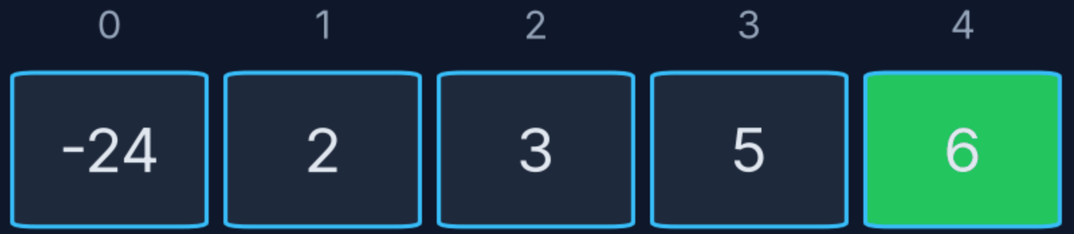


12 → tails[3]

Worked example



smallest tail for each length

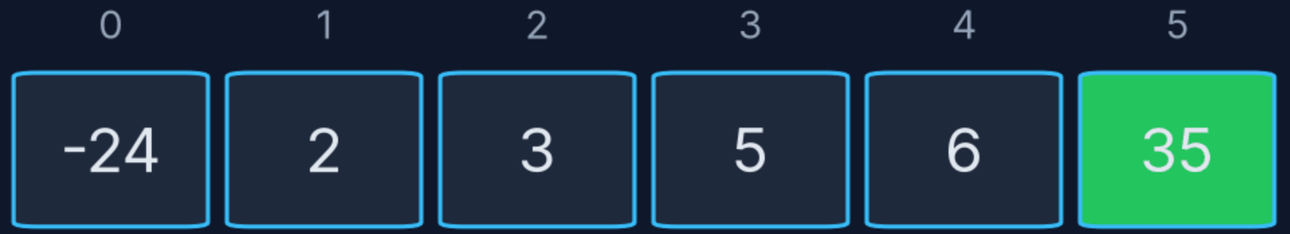


6 → tails[4]

Worked example



smallest tail for each length



35 → tails[5]

Java solution

```java
public class LongestIncreasingSubsequence {
    public static List<Integer> longestIncreasingSubsequence(int[] a) {
        int n = a.length, len = 0;
        int[] tails = new int[n], prev = new int[n];
        for (int i = 0; i < n; i++) {
            int lo = 0, hi = len;
            while (lo < hi) { int m = (lo + hi) >>> 1; if (a[tails[m]] < a[i]) lo = m + 1; else hi = m; }
            prev[i] = lo > 0 ? tails[lo - 1] : -1;
            tails[lo] = i;
            if (lo == len) len++;
        }
        List<Integer> out = new ArrayList<>();
        for (int i = len == 0 ? -1 : tails[len - 1]; i != -1; i = prev[i]) out.add(a[i]);
        Collections.reverse(out);
        return out;
    }
}
```

Complexity

- Time: O(n log n)
- Space: O(n)

Edge cases & pitfalls

- Tails isn't the LIS itself — rebuild with prev
- Strictly increasing → lower bound
- Empty input

Interview tips & follow-ups

- Explain why small tails help

Recap

- Patience sorting: keep the smallest possible tail for each length and binary search where each number goes.
- Complexity: O(n log n)